AI Compiler Performance Engineering

Auto-Loops Books — ch15

Contents

1 XLA: Production-Grade AI Compiler **1**

1.1 XLA HW matrix 1

1.2 XLA arch passes 3

1.3 XLA codegen 7

1.4 XLA tuning 10

1.5 XLA case study 13

1.6 Key Takeaways 17

1.7 Conclusion 17

Bibliography **19**

Index **21**

Chapter 1

XLA: Production-Grade AI Compiler

XLA (Accelerated Linear Algebra) is the production compiler behind JAX and TensorFlow and the only compilation path for TPU, which makes it the most widely deployed ML compiler in this Part—and the one most often dismissed as "just fusion" until decode telemetry says otherwise (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Its design is the reference pattern for multi-hardware AI compilation: frameworks emit StableHLO, XLA converts it to HLO, optimizes, and codegens for CPU, GPU, and TPU from one graph (OpenXLA Project, 2024; Lai *et al.*, 2023). The decode stakes are the book's usual ones: dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 and OLMoE-1B/7B averages 9,305, and XLA can shrink that launch count when its fusion and capture passes are legal, but it cannot fix illegal HBM residency (Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024). The mistake to avoid is tuning XLA on prefill GEMM shapes while batch-1 decode still launches dozens of thin elementwise kernels per token. XLA's automatic fusion is powerful precisely because it is automatic, but "automatic" means it optimizes whatever you profile, so it must be pointed at the decode operating point (Kwon *et al.*, 2023; Davies *et al.*, 2025).

1.1 XLA HW matrix

XLA's support **matrix** is CPU, GPU, and TPU from one front end, and its distinguishing feature is that it is the only compiler path for TPU—there is

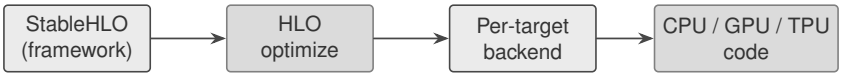


Figure 1.1: XLA lowers one StableHLO graph to HLO, optimizes, and codegens across the CPU/GPU/TPU hardware matrix.

no hand-written-kernel escape hatch on that **target**, so the compiler bears full responsibility (OpenXLA Project, 2024). The portability comes from StableHLO, a versioned, MLIR-based operation set that JAX, TensorFlow, and PyTorch all emit; XLA converts StableHLO to HLO and then sends it to a backend for **target**-specific optimization (OpenXLA Project, 2024; Lai *et al.*, 2023).

The honest reading is that a wide **matrix** is not a uniform decode story. On a **gpu** the backend performs fusions tuned to the GPU programming model and partitions work into streams; on a **cpu** it lowers through LLVM to vectorized native code; on a TPU it is the sole path and applies TPU-specific passes such as bfloat16 propagation (OpenXLA Project, 2024). The same HLO graph therefore gets a different legal schedule per class, and an **npu**-style static fabric is not a native XLA target at all—a reminder that even the broadest production compiler has a shaped, not universal, matrix (AMD, 2024b,a).

Multi-hardware delta. The GPU **matrix** emphasizes fusion plus command-buffer capture; the CPU emphasizes layout and cache blocking; the TPU emphasizes its own partitioning and precision passes—identical HLO, incomparable bytes/token (OpenXLA Project, 2024; SemiAnalysis, 2024). YiRage carries the residency rules as checkable metadata so a chosen **target** schedule is verified decode-legal rather than merely built (Chen and YiRage Contributors, 2026).

1.1.1 Where XLA enters the stack

The production path starts before XLA: a framework exports a program to StableHLO, and a runtime layer hands the module to XLA for compilation and execution (OpenXLA Project, 2024; Lai *et al.*, 2023). JAX lowers every traced function through XLA by default, TensorFlow uses XLA for clusters the compiler decides are profitable or that the user marks with `jit_compile`, and PyTorch reaches XLA through the torch-xla plugin rather than through its default eager path (OpenXLA Project, 2024; Kwon *et al.*, 2023). Each entry point imposes different constraints on what can be compiled: a JAX function is usually captured whole, a TensorFlow cluster can start and stop at arbitrary op boundaries, and torch-xla must export a graph that keeps stateful kernels and sampling outside

the compiled region (Davies *et al.*, 2025; Chen, 2020). Decode teams should know which entry point their serving stack uses because it determines which parts of a token step XLA actually owns and which remain eager.

Runtime execution goes through the PjRt layer, the portable runtime that separates compiled executables from device backends and supports ahead-of-time compilation as well as just-in-time execution (OpenXLA Project, 2024). Ahead-of-time compilation matters for decode serving because model weights, context buckets, and device types are known before requests arrive; compiling once and replaying executable handles removes compile latency from first-token SLOs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Just-in-time compilation, by contrast, trades a compile stall for flexibility and is the default in interactive JAX workloads. A decode product that runs XLA should therefore pin the runtime path, the compiler version, and the compiled executable per model and bucket—otherwise the same source program can silently produce different fusion decisions between cache-hit and cache-miss requests (Chen, 2026; OpenXLA Project, 2024).

1.1.2 StableHLO versioning

StableHLO exists because HLO changed faster than downstream users could track: a versioned, MLIR-based operation set gives frameworks a stable contract to emit against and gives XLA a stable contract to consume (OpenXLA Project, 2024; Lai *et al.*, 2023). For decode teams the versioning contract has a concrete consequence: a model export from a new framework release may target a newer StableHLO version than the serving XLA accepts, and the failure mode is usually not a syntax error but a silently different lowering when the compiler applies compatibility transforms (Chen, 2020; Davies *et al.*, 2025). CI should therefore pin the StableHLO version in the export step and fail loudly on upgrades instead of allowing the runtime to translate between versions. This is the same discipline as pinning CUDA and driver versions in the earlier hardware chapters: the compiler is part of the deployable artifact, not an ambient system library (Chen and YiRage Contributors, 2026; Chen, 2026).

1.2 XLA arch passes

XLA runs hundreds of HLO **pass**es—algebraic simplification, common-subexpression elimination, layout assignment, fusion, scheduling—grouped into pipelines, and fusion is its single most important **optim**ization (OpenXLA Project, 2024). Fusion groups operations into one GPU kernel under a strict invariant: no intermediate

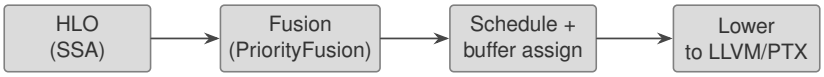


Figure 1.2: XLA architecture passes: hundreds of HLO passes, with fusion, scheduling, and buffer assignment deciding decode residency before target lowering.

is materialized in HBM, everything passes through registers or shared memory, and a fusion compiles to exactly one kernel (OpenXLA Project, 2024). Because decode is memory-bound, this invariant is precisely the bytes/token lever—a fused elementwise chain reads and writes the tensor once instead of once per operator. The fusion decision is made by `PriorityFusion`, a cost-model-driven greedy **pass**, plus Multi-Output fusion for producer-consumer and sibling sharing (OpenXLA Project, 2024).

Two scheduling **pass**es matter for decode residency. The scheduler can "look into the future" because the whole graph is known, choosing an execution order that minimizes peak memory; buffer assignment then produces an optimal allocation, an advantage eager PyTorch or TensorFlow lack since their runtime allocator cannot see the graph ahead (OpenXLA Project, 2024). When peak memory still exceeds capacity, `HloRematerialization` recomputes selected expressions to shorten buffer lifetimes, trading compute for memory—a pass that can cut memory by tens of percent and is required to run many large models (OpenXLA Project, 2024). A `LatencyHidingScheduler` additionally reorders work to overlap compute with communication, which matters for the multi-device decode of Chapter 24, though it can raise peak memory and so interacts with rematerialization (OpenXLA Project, 2024; Davies *et al.*, 2025). Notably XLA **lower**s through StableHLO, an MLIR **dialect**, before its own HLO—so it sits on the same MLIR substrate as IREE while keeping a distinct HLO-centric pipeline (Lai *et al.*, 2023; OpenXLA Project, 2024).

Review gate. Because a "successful" fusion **pass** can still spill if a downstream layout or schedule choice forces it, gate each pass change on buffer accounting at the decode cell: diff the HLO for new allocations and confirm bytes/token did not rise (OpenXLA Project, 2024; Chen, 2026). Silent spill after a green fusion is the classic XLA production surprise (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

1.2.1 Decode-relevant HLO passes

Hundreds of HLO passes sound like complexity, but for decode only a handful determine the counters this book tracks (OpenXLA Project, 2024). The algebraic

simplifier and common-subexpression elimination reduce redundant work and can collapse elementwise chains before fusion ever runs; dead-code elimination removes subgraphs that eager frameworks accidentally retain, such as debug statistics or unused logits (OpenXLA Project, 2024; Chen, 2026). Reshape and transpose passes decide whether a layout change becomes a free metadata operation or a physical copy, which matters at every attention and MLP seam (Davies *et al.*, 2025; SemiAnalysis, 2024). Layout assignment then fixes the physical memory order for every tensor, and buffer assignment turns that layout into concrete allocations whose lifetimes the scheduler can minimize (OpenXLA Project, 2024). A decode engineer who cannot name which of these passes changed between two compiler versions will misattribute the resulting bytes/token regression.

1.2.2 Pass ordering and decode residency

The ordering of layout assignment, fusion, and scheduling is the reason two XLA versions can compile the same StableHLO into different kernels. Fusion runs against the layouts that earlier passes chose, so a transpose inserted before fusion can either enable a larger fused region or strand an operator that needs a different memory order (OpenXLA Project, 2024; Chen, 2020). Schedulers then choose an execution order for the remaining unfused operations, and buffer assignment converts lifetimes into addresses (OpenXLA Project, 2024). Because decode is a long dependency chain over small tensors, small ordering differences amplify into launch counts: an operator placed outside a fusion becomes a separate thunk even when its compute is trivial (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The practical consequence is that compiler upgrades deserve the same decode-cell regression review as kernel rewrites, not just a unit-test green light (Chen, 2026; Chen and YiRage Contributors, 2026).

1.2.3 Rematerialization for long context

`HloRematerialization` trades recomputation for memory: when peak buffer usage exceeds capacity, the pass re-executes selected cheap producers instead of keeping their outputs alive across the whole graph (OpenXLA Project, 2024). For decode the trade is usually favorable for recomputing small activation fragments while keeping large KV-related buffers resident, and usually unfavorable for anything that would double weight reads (Chen, 2026; Kwon *et al.*, 2023). Because rematerialization decisions depend on the cost model, a context-length change can flip which tensors are recomputed and silently raise bytes/token at long context (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Teams serving long-context decode should profile the

rematerialization choices at their longest bucket and pin the compiler version that made the sane choice, documenting it in the runbook (OpenXLA Project, 2024; Chen and YiRage Contributors, 2026).

1.2.4 Reading HLO dumps

XLA can dump the module at pipeline stages, which turns compiler behavior from a black box into a reviewable diff (OpenXLA Project, 2024). The decode workflow should compare HLO before and after fusion, count the remaining top-level computations, and inspect buffer assignment for tensors that grew unexpectedly (Chen, 2020; Davies *et al.*, 2025). A fusion that succeeded in the HLO dump but still spills at runtime points to a scheduling or allocation interaction that the dump alone cannot show; that is the moment to profile DRAM traffic rather than read more IR (Chen, 2026; Vellaisamy *et al.*, 2026). Compiler teams should archive dumps with the benchmark artifacts so a regression can be bisected to the exact pass change (Chen and YiRage Contributors, 2026; Chen, 2020).

1.2.5 Fusion legality in practice

Fusion legality in XLA is not a single check but a sequence of constraints that must all hold. The fused operations must agree on shapes and layouts, no operation inside the fusion may require an HBM round trip such as a collective or a custom call, and the resulting kernel must fit the target's on-chip budget once tiling is chosen (OpenXLA Project, 2024; Chen, 2020). If any constraint fails, the pass either leaves the operator unfused or splits the fusion at the offending edge (OpenXLA Project, 2024; Chen, 2026). Decode graphs expose these constraints more sharply than prefill graphs because thin elementwise chains sit between memory-bound GEMMs, so a layout mismatch that would cost little at large batch can strand a chain and add a launch per token (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The production lesson is to inspect which fusions XLA actually formed at the decode shape rather than assuming the pass summary applies to every shape the team compiles (Davies *et al.*, 2025; Chen, 2026).

1.2.6 Multi-output fusion and shared operands

Multi-output fusion lets one kernel produce several results that different consumers need, which matters when attention output feeds both a residual branch and a normalization path (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026). Without

multi-output fusion the compiler would either duplicate work or write an intermediate once per consumer; with it, one fused kernel produces the shared values in registers or shared memory (OpenXLA Project, 2024; Chen, 2026). Sibling fusion extends the same idea to operations that do not consume each other but share a producer, grouping them so the shared producer executes once (OpenXLA Project, 2024; Chen, 2020). Decode code review should look for these structures in the HLO dump because they are the XLA-native form of the boundary elimination described in earlier chapters (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). When multi-output fusion disappears between compiler versions, residual-add chains quietly regain an HBM round trip (Chen, 2026; Davies *et al.*, 2025).

1.3 XLA codegen

Codegen in XLA is per backend. The CPU and GPU **backend**s use LLVM for low-level IR and code generation: the CPU path **emit**s LLVM IR lowered to x86/ARM native code, and the GPU path **emit**s NVPTX IR lowered to PTX and then SASS (OpenXLA Project, 2024). For more advanced fusions involving matmul or softmax, XLA:GPU uses Triton as a code-generation layer—HLO fusions are converted to TritonIR, tiling parameters are selected, and Triton emits the PTX—so XLA and the Triton compiler of Chapter 17 share a codegen path (OpenXLA Project, 2024; Tillet *et al.*, 2019). For operations better served by vendor libraries, XLA pattern-matches to cuBLAS or cuDNN calls instead of codegen (OpenXLA Project, 2024).

The decode-critical codegen feature is launch capture. XLA:GPU groups a sub-sequence of compatible thunks into a `CommandBufferThunk` that records a GPU command buffer on first execution and replays it on every subsequent call, avoiding the CPU overhead of launching each kernel individually—structurally the same win as the CUDA Graphs of Chapter 8, built into the compiler (OpenXLA Project, 2024; NVIDIA, 2024). This is how XLA attacks launches/token: a decode step that eager execution issues as many launches becomes one replayed command buffer (Vellaisamy *et al.*, 2026; Chen, 2026). The **platform** caveat is that capture requires static shapes and no illegal sync points, so a dynamically-shaped decode loop must be bucketed to stay capturable.

1.3.1 From HLO to thunks

The GPU backend lowers each top-level HLO computation into a sequence of thunks—executable units that know how to launch one piece of work on the device

Backend	Codegen at decode
GPU	LLVM/NVPTX + Triton emitters; command-buffer capture
CPU	LLVM to x86/ARM; cache blocking, vendor libs
TPU	XLA-only path; partitioning + bfloat16 passes

Table 1.1: XLA codegen focus per target at batch-1 decode.

(OpenXLA Project, 2024). A fused computation becomes one thunk; an unfused graph becomes a stream of thunks, one per remaining operation (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026). This mapping is why fusion is the bytes lever and command buffers are the launch lever: fusion reduces the number of thunks, while command buffers reduce the host cost of submitting the thunks that remain (Chen, 2026; NVIDIA, 2024). When reading an XLA profile, the thunk list is a better object than the original framework graph because it is exactly what the device executes. Decode teams should be able to answer how many thunks one token step produces after fusion and how many of those are replayable from a captured command buffer (Davies *et al.*, 2025; Chen, 2020).

1.3.2 Command-buffer capture semantics

The `CommandBufferThunk` records GPU work on first execution and replays it on later calls, but capture semantics matter more than the name suggests (OpenXLA Project, 2024; NVIDIA, 2024). Capture requires stable shapes, stable buffer addresses, and no operations that synchronize illegally or allocate inside the recorded region (OpenXLA Project, 2024; Chen, 2026). A dynamic KV length therefore forces either a shape bucket boundary or a fallback execution path; the compiler should make that fallback explicit so telemetry can count how often a decode step runs uncaptured (Kwon *et al.*, 2023; Davies *et al.*, 2025). Capture also does not change arithmetic: replaying a captured sequence of thunks removes dispatch overhead but leaves every fusion boundary exactly where the earlier passes placed it (NVIDIA, 2024; Chen, 2026). This separation is why XLA can simultaneously win on launches/token through capture and still lose on bytes/token if fusion was never legal at the decode shape.

1.3.3 Triton emission boundaries

XLA:GPU routes selected fusions through Triton rather than lowering everything through LLVM directly (OpenXLA Project, 2024; Tillet *et al.*, 2019). The compiler

chooses the Triton path when a fusion contains matmul or softmax patterns that benefit from tiled code generation, and it autotunes tile parameters as part of the compilation (OpenXLA Project, 2024; Davies *et al.*, 2025). Because Triton emission introduces its own layout and tiling choices, the same fusion can codegen differently across GPU SKUs; a config that wins on one shared-memory budget can spill on another (NVIDIA, 2022; Chen, 2026). Decode teams fingerprinting XLA executables should therefore include the Triton autotune results in the artifact fingerprint, because two builds with identical HLO can still differ in emitted PTX (Chen and YiRage Contributors, 2026; Chen, 2020).

1.3.4 Vendor library and custom-call boundaries

XLA does not codegen every operation itself: large GEMMs and convolutions pattern-match to cuBLAS or cuDNN, and custom operations enter through custom calls when no native HLO lowering exists (OpenXLA Project, 2024). Each boundary is a potential fusion stop: a custom-call kernel cannot be fused through by the generic passes unless the backend knows its semantics (Chen, 2020; Davies *et al.*, 2025). FlashAttention-style custom kernels therefore sit at a boundary where XLA can orchestrate around them but cannot legally fuse through them (Dao *et al.*, 2022; OpenXLA Project, 2024). The engineering takeaway is to count custom-call boundaries in the HLO dump: a decode graph whose attention is one custom call and whose MLP is fused may still be launch-efficient, but its residency story ends where the custom call begins (Vellaisamy *et al.*, 2026; Chen, 2026).

1.3.5 CPU and TPU decode codegen

The CPU backend lowers HLO to LLVM IR and then to native code, which makes XLA a legitimate baseline for CPU decode serving rather than only a GPU tool (OpenXLA Project, 2024; Goodfellow *et al.*, 2016). Cache blocking, vectorization width, and thread scheduling are the CPU analogs of GPU tiling, and XLA's known-graph advantage applies there too: buffer assignment can keep an activation chain resident across operations that an eager runtime would spill between kernel boundaries (Goodfellow *et al.*, 2016; OpenXLA Project, 2024). On TPU, XLA is the only path, so every residency decision is a compiler decision; precision propagation passes such as bfloat16 handling and the TPU partitioning logic become part of the decode story (OpenXLA Project, 2024; Davies *et al.*, 2025). Teams that target TPU decode should therefore treat XLA version pinning as more critical than on GPU, where a fallback eager or Triton path exists (Davies *et al.*, 2025; OpenXLA Project, 2024). The chapter-level lesson stands across all

three: the compiler's automated choices are excellent until the operating point changes, so the artifact and its profile must be versioned together (Chen, 2026, 2020).

1.3.6 Streams, collectives, and multi-device decode

XLA partitions work across streams and devices when the graph contains collectives, which matters for tensor-parallel and pipeline-parallel decode even though this book keeps its core analysis at batch-1 single device (OpenXLA Project, 2024; Davies *et al.*, 2025). The latency-hiding scheduler overlaps communication with compute, and its decisions can change buffer lifetimes in ways that raise peak memory (OpenXLA Project, 2024; Chen, 2026). For decode, the risk is that a scheduler tuned for throughput parallelism increases per-token latency by delaying a dependent attention step behind a collective (Davies *et al.*, 2025; Liu *et al.*, 2024). Multi-device XLA configurations therefore need their own mode matrix entry: the same model at the same context bucket can have a different optimal schedule for TP=1 decode and TP=4 decode (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Compile the partitioned graph, not the single-device graph, when benchmarking fleet configurations (Chen, 2020; OpenXLA Project, 2024).

1.4 XLA tuning

XLA's **tun**ing is mostly automatic, which is its defining contrast with TVM: where TVM searches a large schedule space with AutoTVM and Ansor, XLA relies primarily on heuristics and a cost model and exposes autotune flags for a narrower set of choices such as GPU tiling for Triton-emitted fusions (OpenXLA Project, 2024; Chen *et al.*, 2018). This makes XLA low-effort—a JAX or TensorFlow user benefits without writing schedules—but it also means the engineer's leverage is in **profile**-guided choices and flags rather than in writing kernels (OpenXLA Project, 2024; Davies *et al.*, 2025).

The **pitfall** is the operating point, as with every compiler in this Part. XLA optimizes whatever you compile and **profile**, so compiling on prefill-shaped, large-batch graphs produces fusions and layouts tuned for throughput that break at batch-1 decode, where the GEMM is a matrix-vector product and the launch floor dominates (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The fix is mechanical: compile and **profile** at the decode shape (BS=1, realistic context), decompose ms/token into bytes/token and launches/token, and only then trust the autotuned result (Davies *et al.*, 2025; Chen, 2026). The right configuration is **hardware-**

specific—a Triton-emitted fusion's tiling that fits one GPU's shared memory does not transfer to another—so re-tune per SKU and fingerprint configs so a fleet rollback replays the same HLO snapshot (NVIDIA, 2022; OpenXLA Project, 2024).

A second **pitfall** appears on mixture-of-experts models. MoE graphs have data-dependent routing, which both enlarges XLA's autotune search and risks the compiler choosing layouts that page experts inefficiently if it is not given expert-aware constraints (Liu *et al.*, 2024; Davies *et al.*, 2025). Because XLA leans on heuristics rather than exhaustive search, a poorly-constrained MoE compilation can settle on a schedule that looks fine on a dense **profile** but thrashes when real routing skews load across experts (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The practical discipline is to cap autotune wall time, constrain the search with model-structure hints, and require a decode-cell replay on realistic routing before promoting any winning config to the fleet—the same regression gate Chapter 24 applies across targets.

1.4.1 The tuning surface

XLA exposes a smaller tuning surface than search-based compilers, but the surface still spans several coupled decisions: which fusions Triton emits, what tile shapes those fusions use, whether command buffers capture a given thunk sequence, and whether rematerialization or the latency-hiding scheduler reorders work (OpenXLA Project, 2024; Davies *et al.*, 2025). Each knob has a compiler flag or configuration entry, and each interacts with the others, so tuning one in isolation usually produces a locally better compile and a globally worse decode profile (Chen, 2020, 2026). The right unit of tuning is the full executable artifact compiled for one model, one context bucket, and one GPU SKU, fingerprinted with the HLO dump and the flags that produced it (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Teams that tune flags in production without fingerprinting cannot roll back a regression to a known-good configuration.

1.4.2 Decode-cell tuning workflow

The workflow that keeps XLA honest starts with an eager or cuDNN baseline at the exact decode shape, then compiles the same model through XLA and compares three counters: ms/token, launches or thunks per token, and bytes/token (Davies *et al.*, 2025; Chen, 2026). If XLA wins on latency but bytes/token is flat, the win is orchestration (command buffers), not residency; if bytes/token drops, fusion is doing the work the book cares about (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022). Each candidate configuration should be evaluated at multiple context buckets

because a tile chosen for 2K context can spill or underutilize at 32K (Kwon *et al.*, 2023; SemiAnalysis, 2024). The workflow belongs in CI with the benchmark artifact stored next to the compiler version, so a future XLA upgrade is treated as a change to be evaluated, not an ambient dependency to accept (Chen, 2020; Chen and YiRage Contributors, 2026).

1.4.3 Bucketing dynamic decode shapes

Dynamic shapes are the main obstacle to both Triton tiling stability and command-buffer capture (OpenXLA Project, 2024; Kwon *et al.*, 2023). XLA handles dynamism either by padding to a static shape or by recompiling per shape, and both paths cost bytes or latency (OpenXLA Project, 2024; Davies *et al.*, 2025). Decode serving should therefore bucket context lengths—for example at powers of two with a safety margin—and compile one executable per bucket (Vellaisamy *et al.*, 2026; Chen, 2026). The bucket set is a product decision: too few buckets waste capacity on padding, too many buckets multiply compile time and graph memory (Kwon *et al.*, 2023; Chen, 2020). Telemetry should report which bucket each request used and how often padding exceeded a configured bound, because padding waste is invisible in a profile that only reports ms/token (Davies *et al.*, 2025; Chen, 2026).

1.4.4 A/B methodology across XLA versions

Compiler upgrades deserve the same A/B discipline as kernel rewrites. The minimal A/B pins the model checkpoint, tokenizer, weights, context bucket, driver, and both compiler versions, then compares the three decode counters on identical inputs (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Because XLA fusion output changes with layout-assignment heuristics, a semantic no-op upgrade can still produce different kernels; the HLO dumps make that visible before fleet rollout (OpenXLA Project, 2024; Chen, 2020). Release notes for a decode stack should therefore list the XLA commit and the byte-sheet delta, not merely "upgraded XLA" (Chen and YiRage Contributors, 2026; Chen, 2026). When the delta is unfavorable, the runbook needs a known-good compiled artifact to fall back to, which is why executable fingerprinting is a deployment requirement rather than a profiling nicety (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026).

1.4.5 Routing-aware compilation for MoE

MoE decode adds a data-dependent axis that XLA's heuristics must account for: the router selects a subset of experts per token, and the compiler cannot know the

selection statically (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). XLA handles such graphs either by padding all experts into a static layout or by keeping routing logic at a boundary where the runtime can choose (OpenXLA Project, 2024; Davies *et al.*, 2025). Static padding is simple and capturable but pays full expert bytes even when the router selects a fraction of them; a runtime-selected path saves bytes but complicates command-buffer capture (Kwon *et al.*, 2023; Chen, 2026). The tuning consequence is that MoE decode cannot be tuned on dense-shaped profiles: the configuration must be evaluated with realistic routing distributions at the deployment batch and context (Liu *et al.*, 2024; Davies *et al.*, 2025). Compile-time hints that separate the router from the expert body give XLA the structure it needs without hand-writing kernels, which is the middle path this book recommends for MoE serving (Chen, 2020; Chen and YiRage Contributors, 2026).

1.5 XLA case study

The clearest **case** is an activation like GELU. In eager execution a GELU expands into seven or more separate kernel launches—each elementwise op its own kernel, each writing its intermediate to HBM; XLA fuses the entire GELU into one kernel that reads its input once and writes its output once (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026). **Benchmark**ed at the decode operating point, the win is not a faster multiply but the removal of six HBM round trips and six launches, exactly the bytes/token and launches/token levers the book tracks (Chen, 2026; Dao *et al.*, 2022). Extending the same workflow—compile with fusion, diff the HLO before and after, validate command-buffer capture on the fused decode step—turns "XLA is faster" into a measured, reproducible claim.

The **cross-hardware** reading is where XLA's breadth helps and stops. Because the same StableHLO export compiles to CPU (LLVM, cache-blocked), GPU (LLVM/Triton, captured), and TPU (XLA-only), a team can A/B the identical graph across targets on one decode-cell methodology and compare counters rather than FLOPs (OpenXLA Project, 2024; Davies *et al.*, 2025). But XLA does not target static NPU fabrics, and its automatic fusion can miss residency a search-based or hand-fused kernel would find, so it is one strong tool in the matrix, not the universal answer (AMD, 2024b; Wu *et al.*, 2024). **YiRage** encodes the residency rules XLA's passes leave implicit as checkable IR metadata, so a fusion that would spill is rejected at compile time across backends rather than discovered in production (Chen and YiRage Contributors, 2026).

1.5.1 Worked decode step: one transformer layer

Trace one Llama-class decode layer as XLA would compile it. The framework emits a StableHLO graph containing the QKV projection, score GEMM, softmax, attention output projection, residual add, normalization, and the MLP up/gate/down projections (Dao *et al.*, 2022; Kwon *et al.*, 2023; Davies *et al.*, 2025). In an eager serving stack, the same subgraph often becomes more than a dozen host-visible kernels with HBM round trips between them, which is the regime TaxBreak counts as hundreds of launches per output token on dense models (Vellaisamy *et al.*, 2026; Chen, 2026). XLA's elementwise and multi-output fusion can collapse the projections, residual, and norm chains into a small number of fused computations, and command-buffer capture then replays those computations without per-kernel dispatch (OpenXLA Project, 2024; NVIDIA, 2024). The measured claim is the important one: the layer is shippable only if the compiled executable shows lower launches/token and no higher bytes/token than the eager baseline at the same context bucket (Chen, 2026; Davies *et al.*, 2025).

1.5.2 What fusion does not fix in the worked step

The same trace exposes XLA's boundary. If attention is a custom call or if the softmax path materializes its probability tensor because the emitted graph was not shaped for online softmax, then fusion stops at that boundary and bytes/token stays high no matter how many elementwise chains collapse around it (Dao *et al.*, 2022; OpenXLA Project, 2024; Chen, 2026). A second boundary appears at the MLP seam: fused attention output written to HBM before the following GEMM reintroduces a full-width round trip that no capture win removes (Chen, 2026; Vellaisamy *et al.*, 2026). The review gate for the worked step is therefore the HLO diff plus the DRAM counter, not the compiled-binary existence: an executable that compiles and replays can still encode the very round trips the decode optimization promised to remove (Chen, 2020; Davies *et al.*, 2025). Engineers should treat those cases as evidence that the source graph needs restructuring—for example exporting online softmax or a fused attention region—rather than as evidence that XLA is slow (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

1.5.3 Replicating the case study

The case study is reproducible from any framework XLA supports. From JAX, write the layer with explicit shapes, compile at the decode bucket, and inspect the executable plus HLO dump; from TensorFlow, mark the cluster for compilation

and profile the fused program; from torch-xla, move the decode loop into a traced context and compare against the eager baseline (OpenXLA Project, 2024; Davies *et al.*, 2025). Every replication should record the model checkpoint, context bucket, compiler version, and the three counters before the experiment starts, because those four fields are what make the result comparable across teams (Chen, 2026; Vellaisamy *et al.*, 2026). The same replication script should run on CPU and GPU locally and, where available, on TPU, turning the XLA matrix claim into a triplet regression instead of a vendor slide (OpenXLA Project, 2024; Chen, 2020). A decode team that cannot reproduce the GELU-level win on its own model should suspect a boundary in the exported graph, not the compiler.

1.5.4 Promotion worksheet

Before promoting an XLA-compiled decode executable, confirm the worksheet has five entries: the exact StableHLO and HLO versions, the bucket set with measured padding waste, the Triton autotune and command-buffer configuration fingerprint, the three-counter comparison against the eager baseline at every bucket, and a known-good artifact for rollback (OpenXLA Project, 2024; Chen, 2026; Davies *et al.*, 2025). Each entry should be machine-generated from the build so the worksheet cannot drift from what actually shipped (Chen and YiRage Contributors, 2026; Chen, 2020). If any entry is missing, the promotion is a gamble that will be resolved later by a fleet incident (Vellaisamy *et al.*, 2026; Chen, 2026). This worksheet is the XLA instance of the general compiler-deployment contract the book repeats: pin, profile, diff, and keep the rollback path warm (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

1.5.5 MoE scenario and launch collapse

Apply the case-study method to an OLMoE-class MoE model whose eager profile shows thousands of launches per output token (Vellaisamy *et al.*, 2026). The XLA compile should first collapse the dense attention and shared layers with fusion and capture; the expert body then becomes the remaining tuning question (OpenXLA Project, 2024; Davies *et al.*, 2025). If the router and expert gather-scatter stay outside the captured region, the serving stack keeps an eager hull around a captured core—the hybrid pattern from Chapter 12—and the measured win is a large launch reduction in the shared portion (NVIDIA, 2024; Vellaisamy *et al.*, 2026). The byte ledger must still show expert weights and activations moving efficiently, because a launch collapse that pads every expert into the graph can increase bytes/token even as latency improves (Chen, 2026; Kwon *et al.*, 2023). Publish the MoE scenario

with the same counters as the dense scenario so fleet reviewers can compare like for like (Davies *et al.*, 2025; Chen, 2020).

1.5.6 Failure autopsies for XLA decode

Three failure signatures recur in XLA decode rollouts, and each maps to a different cause. Flat latency with rising DRAM traffic points to fusion boundaries that did not hold at the decode shape, visible in the HLO dump as unfused seams around attention or residual chains (Chen, 2026; OpenXLA Project, 2024). First-token latency spikes after a warm fleet point to cache misses, recompilation, or command-buffer re-capture triggered by a shape or allocator change (Davies *et al.*, 2025; NVIDIA, 2024). A configuration that wins on one GPU and loses on another points to Triton tile autotune results that did not transfer, which the artifact fingerprint should have caught before promotion (OpenXLA Project, 2024; NVIDIA, 2022). Each autopsy should start from the pinned executable and HLO dump, not from framework-level timers, because the compiler artifacts are the only record of what actually ran (Chen, 2020; Chen and YiRage Contributors, 2026). Teams that archive those artifacts turn the third failure into a config diff and the first two into pass-ordering bugs instead of folklore (Chen, 2026; Vellaisamy *et al.*, 2026).

1.5.7 Serving XLA decode in production

The operational layer around XLA matters as much as the compiler passes. Every serving release should record the compiler commit, StableHLO version, autotune fingerprint, and compiled executable hash in the same artifact store as the model weights (OpenXLA Project, 2024; Davies *et al.*, 2025; Chen, 2026). Startup should load precompiled executables for the shipped bucket set rather than compiling on first request, and telemetry should expose bucket hit rate, padding waste, command-buffer replay rate, and fallback-to-eager rate as first-class metrics (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2020). On-call runbooks need a one-command rollback to the previous known-good executable, not a recompile under incident pressure (Chen and YiRage Contributors, 2026; Chen, 2026). Teams that treat XLA as part of their deployable stack get deterministic decode; teams that treat it as ambient infrastructure get whichever compile the cache happened to hold (Davies *et al.*, 2025; OpenXLA Project, 2024). The same principle applies to Triton-emitted fusions: their tile decisions belong to the shipped artifact and must be reproducible in CI, or the next compile can silently change decode behavior without any model change (OpenXLA Project, 2024; Davies *et al.*, 2025).

1.6 Key Takeaways

- **XLA is automatic, broad, and TPU's only path.** StableHLO → HLO → LLVM/PTX/TPU compiles one graph to CPU, GPU, and TPU without hand-written kernels (OpenXLA Project, 2024; Lai *et al.*, 2023). Automatic means it optimizes whatever you profile, so point it at decode (Davies *et al.*, 2025).
- **Fusion is the bytes/token lever.** XLA's fusion invariant—no intermediate in HBM, one fusion equals one kernel, driven by PriorityFusion—is exactly the on-chip residency decode needs (OpenXLA Project, 2024; Chen, 2026). Gate each fusion pass on buffer accounting (Vellaisamy *et al.*, 2026).
- **Command-buffer capture is the launches/token lever.** `CommandBufferThunk` records and replays a GPU command buffer, removing per-kernel host dispatch like CUDA Graphs, but requires static shapes (OpenXLA Project, 2024; NVIDIA, 2024). Bucket dynamic decode shapes to stay capturable (Kwon *et al.*, 2023).
- **Codegen shares Triton and LLVM.** CPU/GPU use LLVM; advanced GPU fusions emit through TritonIR, so XLA and Triton share a codegen layer (OpenXLA Project, 2024; Tillet *et al.*, 2019). Vendor-library pattern-matching handles large GEMM/conv (NVIDIA, 2022).
- **Tune at the decode operating point.** XLA's heuristics optimize the shape you compile, so prefill-shaped tuning breaks batch-1 decode; profile BS=1 ms/token and fingerprint configs per SKU (Davies *et al.*, 2025; OpenXLA Project, 2024). Re-tune per hardware (NVIDIA, 2022).

1.7 Conclusion

XLA shows that automatic, whole-graph compilation is production-viable at the largest scale: StableHLO portability, HLO fusion that keeps intermediates off HBM, scheduling that exploits the known graph, command-buffer capture that removes launch overhead, and LLVM/Triton codegen across CPU, GPU, and TPU (OpenXLA Project, 2024; NVIDIA, 2024). Its automation is its strength and its trap—it optimizes the operating point you give it—so the decode discipline is to compile and grade at batch-1 on bytes/token and launches/token, the same residency contract YiRage enforces as metadata (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). With XLA's automatic approach established, Chapter **??**

turns to TVM, which pursues the same multi-hardware goal through explicit search rather than heuristics, trading XLA's low effort for a larger reachable schedule space (Chen *et al.*, 2018; Zheng *et al.*, 2020).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 2, 13

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018). TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594. Graph- and operator-level optimization, operator fusion, memory latency hiding, ML-based cost model; CPU/GPU/FPGA. 10, 18

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 2, 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 7, 9, 11, 13, 14

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 9

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 5, 8, 10, 11, 12, 13, 14, 15, 16, 17

Lai, R., Shao, J., Feng, S., Jin, Y., *et al.* (2023). Relax: Composable abstractions for end-to-end dynamic machine learning. *arXiv preprint arXiv:2311.02103*. TVM Unity; cross-level IRModule (Relax graph + TensorIR), first-class symbolic shapes, call_tir DPS; replaces Relay. 1, 2, 3, 4, 17

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 10, 11, 13

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 9, 11, 16, 17

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 7, 8, 14, 15, 16, 17

OpenXLA Project (2024). XLA: Accelerated linear algebra — OpenXLA architecture. `https://openxla.org/xla/architecture`. StableHLO → HLO → LLVM/PTX/TPU; operator fusion as primary optimization (no HBM intermediates); PriorityFusion / Multi-Output passes; CommandBufferThunk launch capture; Triton emitters. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 2, 5, 12

Tillet, P., Kung, H. T., and Cox, D. (2019). Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL@PLDI)*, pages 10–19. Tile = statically shaped sub-array; Triton-C + LLVM-based Triton-IR; matmul/conv on par with cuBLAS/cuDNN. 7, 8, 17

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. (2024). Mirage: A multi-level superoptimizer for tensor programs. *arXiv preprint arXiv:2405.05751*. μGraph kernel/block/thread levels; joint algebraic+schedule+custom-kernel superoptimization; abstraction-based pruning; probabilistic equivalence via PIT over finite fields (LAX); 1.1–2.9×. 13

Zheng, L., Jia, C., Sun, M., Wu, Z., Yu, C. H., Haj-Ali, A., Wang, Y., Yang, J., Zhuo, D., Sen, K., Gonzalez, J. E., and Stoica, I. (2020). Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems*

Design and Implementation (OSDI), pages 863–879. Template-free auto-scheduler; hierarchical sampling, evolutionary search, task scheduler; up to 3.8x/2.6x/1.7x on Intel CPU/ARM CPU/NVIDIA GPU. 18

Index

HLO, 1

StableHLO, 1

XLA, 1